Building LLMs from Scratch

Generated by Bookify

Table of Contents

Introduction

The world of Large Language Models (LLMs) is rapidly evolving, offering unprecedented capabilities in understanding and generating human language. Yet, harnessing their full potential often requires more than just calling an API; it demands a deep understanding of how to adapt these powerful, general-purpose models to specific, real-world tasks. This book embarks on a practical journey to demystify the process of building and customizing LLMs from the ground up, beginning with the crucial technique of **LLM Fine-tuning for Classification**. You'll learn how to take pre-trained models and sculpt them into highly accurate, task-specific tools, transforming abstract language understanding into concrete, actionable insights for diverse applications.

This book is crafted for machine learning engineers, data scientists, researchers, and advanced students who possess a foundational understanding of Python and core machine learning concepts. If you're eager to move beyond high-level abstractions and dive into the practical mechanics of LLM development, this is your guide. By the end of this journey, you won't just know *how* to fine-tune an LLM; you'll understand the underlying principles, the nuances of data preparation, the art of hyperparameter tuning, and robust evaluation strategies. You will gain the confidence and skills to tackle complex classification problems, adapting state-of-the-art models to your unique datasets and business challenges.

Designed as a hands-on companion, this book emphasizes practical application through clear, step-by-step explanations and extensive code examples. Each chapter builds upon the last, guiding you through the entire fine-tuning pipeline, from selecting the right base model and preparing your data to training, evaluating, and deploying your specialized LLM. We encourage you to actively engage with the code, experiment with the concepts, and apply them to your own projects. Prepare to transform your theoretical knowledge into tangible, deployable LLM solutions, empowering you to build intelligent systems that truly understand and interact with the world.

LLM Fine-tuning for Classification

LLM Fine-tuning for Classification: Model Architecture

Fine-tuning a pre-trained Large Language Model (LLM) is crucial for adapting it to specific downstream tasks, such as text classification. This process leverages the extensive general language understanding acquired during pre-training and then specializes the model for a new objective. This section details the steps involved in preparing an LLM architecture for a binary text classification task, specifically identifying spam messages.

As introduced in prior sections, the initial phase of this project involved data acquisition and preprocessing. The SMS Spam Collection dataset [https://archive.ics.uci.edu/dataset/228/sms+spam+collection] was downloaded, and its classes (spam and "ham" for non-spam) were balanced to ensure an equal number of samples (747 each) for robust training [Video: "Coding the model architecture for LLM classification fine-tuning" @ 01:14]. The text messages were then tokenized using a byte-pair encoder, padded to a uniform length of 120 tokens (based on the longest message in the dataset), and organized into input and target tensors for batch processing via data loaders [Video: "Coding the model architecture for LLM classification fine-tuning" @ 01:14]. With the data prepared, the next step is to configure and modify the LLM architecture.

1. Model Initialization with Pre-trained Weights

To avoid the immense computational cost and resources required for training an LLM from scratch, the process begins by initializing a GPT-like model architecture with pre-trained weights. OpenAI has made the weights for various GPT-2 models publicly available, allowing developers to utilize these powerful models as a foundation [Video: "Coding the model architecture for LLM classification fine-tuning" @ 06:18].

For this classification task, the GPT-2 small model, comprising 124 million parameters, is selected. The model's base configuration is set to match the parameters used during GPT-2's original pre-training to ensure compatibility with the loaded weights [Video: "Coding the model architecture for LLM classification fine-tuning" @ 08:03].

Configuration Parameter	Value	Description
Vocabulary Size	50257	Number of unique tokens the model can process.
Context Length	1024	Maximum sequence length the model can handle.
Dropout Rate	0	Dropout probability applied during training.
Query/Key/Value Bias	True	Indicates if bias terms are used in attention mechanisms.

The pre-trained GPT-2 parameters are downloaded and loaded into the custom GPT model architecture. This involves a utility function, `download_and_load_GPT2`, which retrieves the model's configuration settings and its entire parameter dictionary (including token embeddings, positional embeddings, transformer block parameters, and final normalization layer parameters) [Video: "Coding the model architecture for LLM classification fine-tuning" @ 09:39]. These parameters are then systematically loaded into the corresponding layers of the custom GPT model using a `load_weights_into_GPT` function [Video: "Coding the model architecture for LLM classification fine-tuning" @ 12:18]. This effectively "equips" the custom model with the language understanding capabilities of the pre-trained GPT-2.

To verify successful weight loading, the model can be tested for text generation. Providing an input prompt like "Every effort moves you" should result in a coherent and grammatically correct continuation, such as "forward. The first step is to understand the importance of your work" [Video: "Coding the model architecture for LLM classification fine-tuning" @ 14:23]. This confirms that the pre-trained parameters are functioning as expected for their original task of next-token prediction.

However, a pre-trained language model, even with its vast knowledge, does not inherently perform classification. An attempt to perform zero-shot classification by prompting the model with an instruction like "Is the following text spam? Answer with a yes or no: [spam message]" demonstrates its limitation. The model, without specific fine-tuning, struggles to follow such instructions and provide a direct "yes" or "no" answer, instead continuing the text generation task [Video: "Coding the model architecture for LLM classification fine-tuning" @ 15:12]. This highlights the necessity of fine-tuning for task-specific adaptation.

2. Modifying the Architecture for Classification

The core modification for classification fine-tuning involves adapting the model's output layer. In its original form, a GPT model's output layer (often referred to as the "output head") is a linear layer that maps the hidden state representation to a probability distribution over the entire vocabulary (50257 tokens in this case) for next-token prediction [Video: "Coding the model architecture for LLM classification fine-tuning" @ 16:44].

For classification, this output head is replaced with a new linear layer, known as a **classification head**. This new layer maps the model's hidden state (with an embedding dimension of 768) to a smaller number of output units, corresponding to the number of classes in the classification task. For binary spam classification, this means mapping to two units: one for "spam" and one for "no spam" [Video: "Coding the model architecture for LLM classification fine-tuning" @ 18:46]. This approach is generalizable; for a multi-class classification task with N classes, the output head would map to N units.

The architectural change can be visualized as follows:

```
graph TD
    A[Input Tokens] --> B[Token Embeddings]
    B --> C[Positional Embeddings]
    C --> D[Transformer Blocks (12 layers)]
    D --> E[Final Layer Normalization]

    subgraph Original Architecture
        E --> F_orig[Original Output Head (Linear: 768 -> 50257)]
        F_orig --> G_orig[Next Token Probabilities]
    end

    subgraph Modified Architecture for Classification
        E --> F_class[Classification Head (Linear: 768 -> 2)]
        F_class --> G_class[Class Probabilities (Spam/No Spam)]
    end
```

To implement this, all parameters of the pre-trained model are initially "frozen" by setting their `requires_grad` attribute to `False`. This prevents them from being updated during the initial stages of fine-tuning [Video: "Coding the model architecture for LLM classification fine-tuning" @ 27:30]. Then, specific layers are "unfrozen" (or explicitly made trainable) to allow for adaptation to the classification task:

1. **Classification Head**: The `model.output_head` is replaced with a new `nn.Linear` layer that maps the hidden dimension (768) to the number of classes (2). Parameters of this new layer are trainable by default [Video: "Coding the model architecture for LLM classification fine-tuning" @ 28:20].
2. **Final Transformer Block**: The parameters of the last transformer block (`model.transformer_blocks[-1]`) are unfrozen. This allows the model's highest-level feature extraction capabilities to adapt to the nuances of the classification task [Video: "Coding the model architecture for LLM classification fine-tuning" @ 29:41].
3. **Final Layer Normalization**: The parameters of the `model.final_norm` layer, which connects the transformer blocks to the output head, are also

unfrozen [Video: "Coding the model architecture for LLM classification fine-tuning" @ 29:57].

This selective fine-tuning strategy is efficient because the lower layers of the pre-trained LLM (such as token and positional embeddings, and earlier transformer blocks) have already learned fundamental language structures and semantics applicable across a wide range of tasks [Video: "Coding the model architecture for LLM classification fine-tuning" @ 20:46]. By only fine-tuning the upper layers and the newly added classification head, the model can quickly specialize for the new task without retraining its foundational language understanding.

When processing an input sequence, the model generates an output for each token. For classification, only the output corresponding to the **last token** in the sequence is typically used. This is because, due to the self-attention mechanism, the last token's representation has attended to and integrated information from all preceding tokens in the sequence, making it the most contextually rich representation for making a classification decision [Video: "Coding the model architecture for LLM classification fine-tuning" @ 23:57].

For example, if the input is "Do you have time?", the model will produce two output values (for "spam" and "no spam") for each of the four tokens. The output for the "time" token (the last token) is extracted and used for the final classification [Video: "Coding the model architecture for LLM classification fine-tuning" @ 30:45].

Summary

This section detailed the crucial steps in preparing an LLM for classification fine-tuning. It covered initializing the model with pre-trained GPT-2 weights, demonstrating the need for fine-tuning through a zero-shot attempt, and then modifying the architecture by replacing the original output head with a task-specific classification head. Furthermore, it explained the strategic decision to selectively unfreeze and train only the classification head, the final transformer block, and the final layer normalization, leveraging the pre-trained model's general language understanding while adapting it efficiently to the new task. The rationale for using the last output token for classification was also discussed.

With the model architecture now configured and ready, the next phase involves defining the loss function and evaluation metrics, which will enable the training process to begin.

Conclusion

As we reach the final pages of this journey, take a moment to reflect on the incredible path you've traversed. From the foundational principles of neural networks and the intricate dance of attention mechanisms, you've moved beyond theoretical understanding to the practical art of constructing large language models from the ground up. This book has been your guide through the complex landscape of model architecture, data preparation, and the iterative process of training, equipping you with the hands-on experience necessary to transform abstract concepts into functional, powerful AI systems.

You've not only grasped the core mechanics of LLMs but have also mastered the critical skill of adapting these formidable models to specific, real-world challenges. A significant part of our exploration focused on the nuanced process of fine-tuning LLMs for classification tasks, where you learned to leverage pre-trained intelligence to accurately categorize and interpret textual data. This involved understanding data annotation, selecting appropriate loss functions, optimizing hyperparameters, and rigorously evaluating model performance to ensure precision and reliability in diverse classification scenarios. You now possess the expertise to take a general-purpose LLM and sculpt it into a specialized tool capable of solving precise, domain-specific problems with remarkable efficacy.

The knowledge and practical skills you've acquired are not merely endpoints but powerful launchpads for future innovation. The world of LLMs is vast and everevolving, inviting you to explore further into areas like generative fine-tuning for summarization or creative content generation, delving into advanced techniques such as Parameter-Efficient Fine-Tuning (PEFT), or even venturing into the exciting realm of multi-modal models. Continue to experiment, stay curious about new research, and contribute to this rapidly advancing field. You now have the foundational understanding and practical toolkit to not just use, but to build, adapt, and push the boundaries of what's possible with large language models.

Glossary

Classification Head — A new linear layer that replaces the original output head, mapping the model's hidden state to a smaller number of output units corresponding to the number of classes in the classification task.

Context Length — The maximum sequence length the model can handle.

Dropout Rate — The dropout probability applied during training.

Frozen — A state where parameters of the pre-trained model are set with their `requires_grad` attribute to `False`, preventing them from being updated during the initial stages of fine-tuning.

Ham — The non-spam class in the SMS Spam Collection dataset.

Last Token — The output corresponding to the last token in an input sequence, which is typically used for classification because its representation has integrated information from all preceding tokens.

Output Head — A GPT model's original output layer, which is a linear layer that maps the hidden state representation to a probability distribution over the entire vocabulary for next-token prediction.

Parameter Dictionary — A collection of the model's configuration settings and its entire parameter set, including token embeddings, positional embeddings, transformer block parameters, and final normalization layer parameters.

Query/Key/Value Bias — An indicator of whether bias terms are used in attention mechanisms.

Unfrozen — A state where specific layers are explicitly made trainable to allow for adaptation to the classification task.

Vocabulary Size — The number of unique tokens the model can process.

References & Resources

LLM Fine-tuning for Classification

- https://drive.google.com/file/d/1oXbv0T9t74pEXejQ3o7w9ZABGLjHmFsu/view?usp=sharing
- https://archive.ics.uci.edu/dataset/228/sms+spam+collection